

Actividad 1 | Deuda técnica, setup del stack y primer experimento con MLFlow

TC5061 Operaciones de aprendizaje automático

Carlos Humberto Galvan Perales | A01797969

A. Reflexión escrita

Parte A. Reflexión crítica: deuda técnica en sistemas de ML

El paper de Sculley et al. (2015) deja claro un punto que en la práctica se siente tarde: entrenar un modelo suele ser la parte barata; lo caro llega cuando ese modelo toca datos vivos, otros sistemas y gente con plazos. Abajo relaciono tres anti-patrones del paper con situaciones que he visto en trabajo de datos e IA aplicada.

1) Glue code y pipelines

En proyectos de ML el modelo ocupa poco espacio frente al código que mueve datos: lecturas de tablas, joins, limpieza, serialización y despliegue. Eso coincide con lo que el paper llama glue code y, en la preparación de datos, pipeline jungles. En entornos Spark/Databricks es fácil que notebooks y jobs se encadenen “porque ya funcionaba”, con archivos intermedios y pasos difíciles de probar de punta a punta.

- a. Consecuencia: cambiar una librería o un formato de entrada cuesta semanas, y el equipo evita experimentos porque el costo de integración supera el beneficio del modelo. MLOps ayuda aquí al forzar un camino repetible (código versionado, ambiente declarado, registro de corridas) en lugar de otro script suelto.

2) Dependencias de datos inestables o subutilizadas

Sculley insiste en que las dependencias de datos pueden ser peores que las de código porque casi no hay análisis estático. En el ámbito profesional he visto features que vienen de tablas de otro equipo o de un proceso que “mejora” sin avisar. El modelo se ajusta a un sesgo o a un mapeo viejo; el día que corrigen el upstream, el desempeño baja y nadie sabe por qué. También aparecen features legacy o empaquetadas en bloque: aportan poco, pero si alguien las apaga se rompe algo.

- a. Consecuencia: fragilidad y tiempo perdido en diagnóstico. Un flujo MLOps con data contracts, versionado de datasets y leave-one-feature-out periódico reduce exactamente esa deuda.

3) **Consumidores no declarados y feedback loops ocultos**

Cuando un score o una predicción se publica en un log, un dashboard o una API “por si alguien lo necesita”, nacen undeclared consumers. Más adelante un cambio “mejor” al modelo rompe a un consumidor que no estaba en el diseño. En paralelo, si el modelo influye en decisiones operativas (ofertas, priorización de pozos, campañas) y esas decisiones cambian los datos futuros, aparece un feedback loop. Sin monitoreo de sesgo de predicción, límites de acción y dueños claros del contrato de la predicción, el sistema se vuelve difícil de razonar.

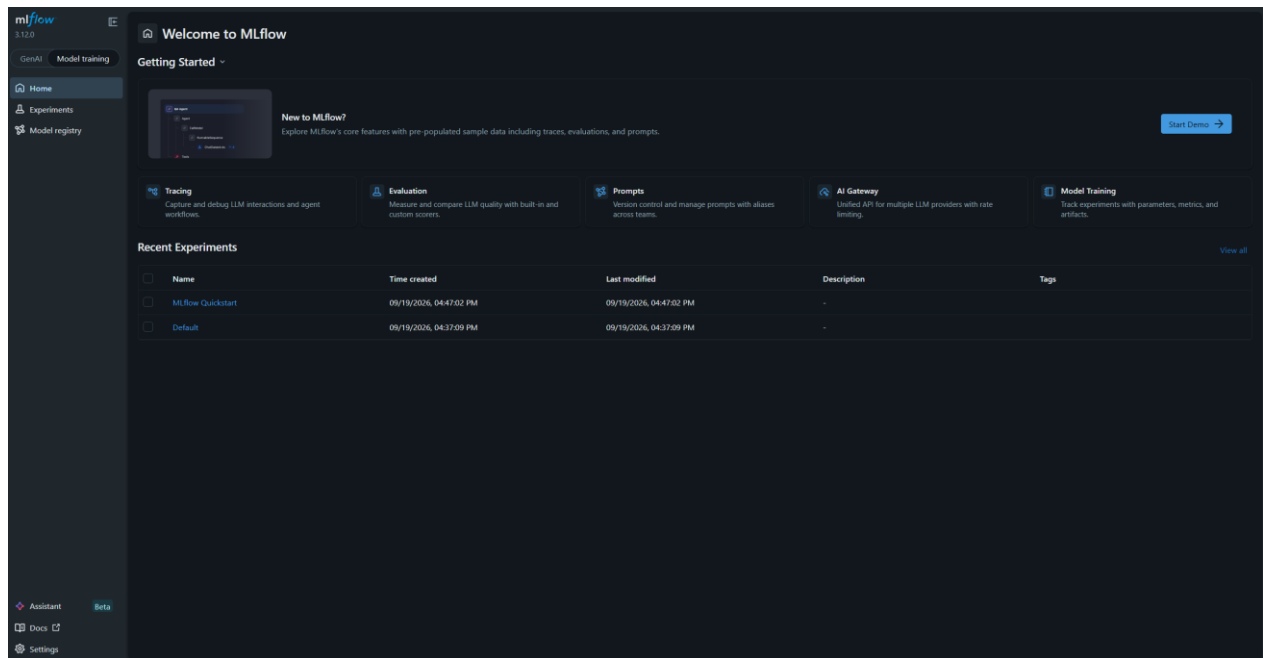
MLOps no elimina el loop, pero lo hace visible: tracking, alertas y ownership.

¿Qué problema concreto resolvería MLOps en ese contexto?

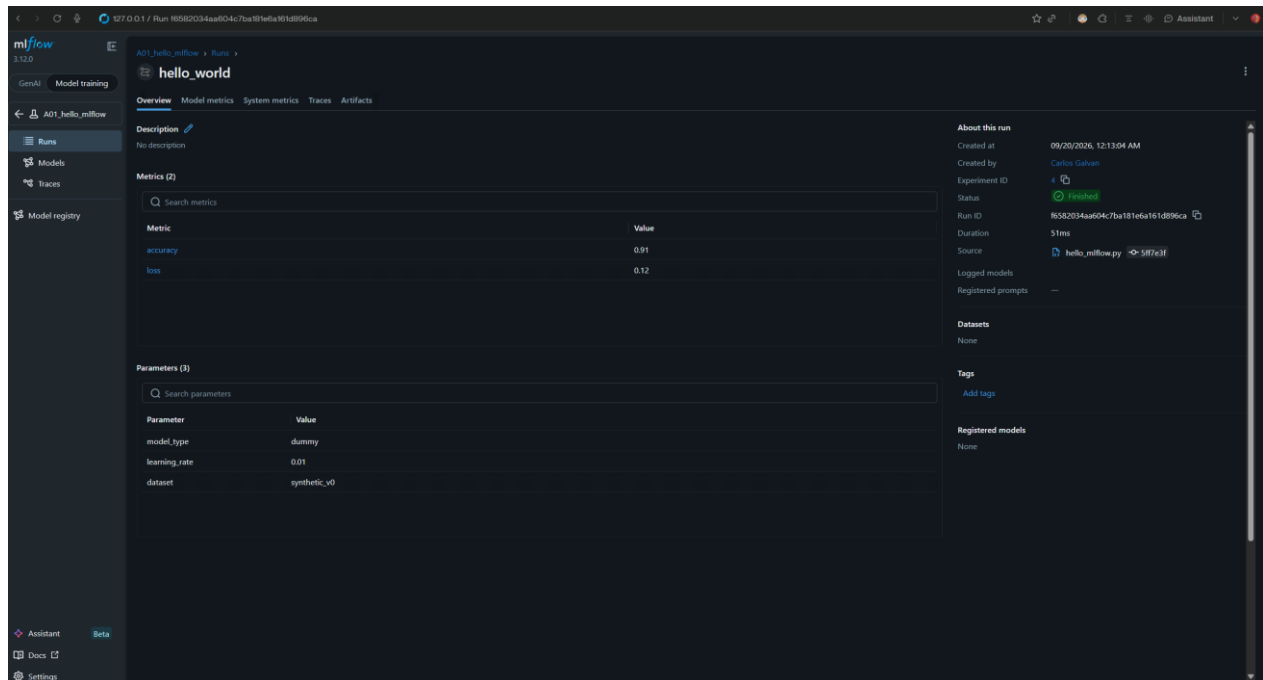
Más allá de “subir el accuracy un punto” se busca poder responder, en producción: qué corrida generó este modelo, con qué datos y parámetros, quién consume la salida, y qué hacer cuando el upstream cambia. En mi día a día la pieza que más ha faltado de forma sistemática es el registro reproducible de experimentos y el cierre del ciclo train, evaluate, register, monitor. Herramientas como MLFlow Tracking atacan esa parte crítica: parámetros, métricas y artefactos en un solo lugar, auditable. Si además se acompaña de buenas prácticas de configuración, borrado de features inútiles y monitoreo live, se paga la deuda de los tres anti-patrones anteriores en lugar de seguir acumulándola.

En resumen, el paper pide tratar el sistema alrededor del modelo con la misma seriedad que el algoritmo. Esa es la lectura que llevo a esta actividad y al resto del curso TC5061.

B. Evidencia de setup del stack (Parte B)



C. Experimento Hello World en MLFlow (Parte C)



Apéndice. Código hello_mlflow.py

```
from pathlib import Path

import mlflow

def main() -> None:

    artifact = Path("hello_artifact.txt")

    artifact.write_text(

        "Primer artefacto A01 TC5061 - Carlos Galvan\n",

        encoding="utf-8",

    )

    mlflow.set_tracking_uri("http://127.0.0.1:5000")

    mlflow.set_experiment("A01_hello_mlflow")

    with mlflow.start_run(run_name="hello_world"):

        mlflow.log_param("model_type", "dummy")

        mlflow.log_param("learning_rate", 0.01)

        mlflow.log_param("dataset", "synthetic_v0")

        mlflow.log_metric("accuracy", 0.91)

        mlflow.log_metric("loss", 0.12)

        mlflow.log_artifact(str(artifact))

    run = mlflow.active_run()

    print(f"Run ID: {run.info.run_id}")

if __name__ == "__main__":

    main()
```